

ECharts 初学者教程

什么是 ECharts

ECharts 是一个前端图表库，可以绘制折线图、柱状图、饼图、散点图、热力图、关系图等。它的核心思想是：用一个 `option` 对象描述图表长什么样、数据从哪里来、如何交互。

本项目将 ECharts 封装在：

```
src/charts/BaseChart.tsx
```

所有图表 `option` 放在：

```
src/charts/options.ts
```

React 如何使用

页面中不直接调用 `echarts.init`，而是使用 `BaseChart`：

```
import BaseChart from '@charts/BaseChart';
import { buildMultiParameterLineOption } from '@charts/options';

export default function Demo({ records }: Props) {
  return <BaseChart option={buildMultiParameterLineOption(records)} height={360}/>;
}
```

`BaseChart` 内部做了这些事：

- 创建 DOM 容器
- 使用 `echarts/core` 初始化，按需注册 `line`、`bar`、`pie`、`grid`、`tooltip`、`legend`、`dataZoom`、`canvas` `renderer`
- `setOption`
- `ResizeObserver` 监听尺寸变化
- 组件卸载时 `dispose`

option 是什么

`option` 是图表配置对象。

```
const option = {
  tooltip: {},
  legend: {},
```

```
xAxis: {},
yAxis: {},
series: [],
};
```

常见字段：

- **title**：标题
- **tooltip**：鼠标悬浮提示
- **legend**：图例
- **xAxis**：X 轴
- **yAxis**：Y 轴
- **dataZoom**：缩放
- **series**：数据系列
- **dataset**：结构化数据集

series 是什么

series 是真正画图的数据。每一个 series 表示一组图形。

折线图：

```
const option = {
  xAxis: { type: 'category', data: ['22:00', '22:15', '22:30'] },
  yAxis: { type: 'value' },
  series: [
    {
      name: 'Heart Rate',
      type: 'line',
      data: [72, 75, 71],
    },
  ],
};
```

柱状图：

```
const option = {
  xAxis: { type: 'category', data: ['apnea', 'hypopnea', 'movement'] },
  yAxis: { type: 'value' },
  series: [
    {
      name: '事件数量',
      type: 'bar',
      data: [4, 7, 3],
    },
  ],
};
```

饼图：

```
const option = {
  series: [
    {
      name: '睡眠阶段占比',
      type: 'pie',
      radius: ['42%', '68%'],
      data: [
        { name: 'N1', value: 5 },
        { name: 'N2', value: 12 },
        { name: 'N3', value: 8 },
      ],
    },
  ],
};
```

xAxis

xAxis 是 X 轴配置。常见类型：

```
// 类目轴，适合时间点、标签
xAxis: {
  type: 'category',
  data: ['22:00', '22:15', '22:30'],
}
```

```
// 数值轴，适合连续数值
xAxis: {
  type: 'value',
}
```

yAxis

yAxis 是 Y 轴配置。

```
yAxis: {
  type: 'value',
  name: 'bpm',
}
```

睡眠阶段这种非数字标签，可以用 **formatter** 转换显示：

```
yAxis: {
  type: 'value',
  min: 0,
  max: 4,
  axisLabel: {
    formatter: (value: number) => {
      const map = ['N3', 'N2', 'N1', 'REM', 'Wake'];
      return map[value] ?? '';
    },
  },
},
}
```

tooltip

tooltip 控制鼠标悬浮提示。

```
tooltip: {
  trigger: 'axis',
}
```

常用值:

- **axis**: 按坐标轴触发, 适合折线图
- **item**: 按单个图形触发, 适合饼图

legend

legend 控制图例。用户可以点击图例隐藏/显示某条曲线。

```
legend: {
  data: ['Heart Rate', 'SpO2'],
}
```

dataZoom

dataZoom 用于缩放和拖拽。

```
dataZoom: [
  { type: 'inside', start: 0, end: 100 },
  { type: 'slider', start: 0, end: 100 },
];
```

- **inside**: 鼠标滚轮或触控板缩放
- **slider**: 底部可拖动缩放条

dataset

dataset 适合表格型数据。它可以减少多个 series 重复取数据。

```
const option = {
  dataset: {
    source: [
      ['time', 'heartRate', 'spo2'],
      ['22:00', 72, 98],
      ['22:15', 75, 97],
    ],
  },
  xAxis: { type: 'category' },
  yAxis: {},
  series: [
    { type: 'line', encode: { x: 'time', y: 'heartRate' } },
    { type: 'line', encode: { x: 'time', y: 'spo2' } },
  ],
};
```

本项目当前直接在 option 中 **map** 数据，更直观，适合 Demo 阶段。

如何新增折线图

1. 定义数据类型。

```
interface HeartRateRecord {
  time: string;
  heartRate: number;
}
```

2. 写 option 构造函数。

```
export const buildHeartRateOption = (records: HeartRateRecord[]) => ({
  tooltip: { trigger: 'axis' },
  xAxis: {
    type: 'category',
    data: records.map((record) => record.time),
  },
  yAxis: {
    type: 'value',
    name: 'bpm',
  },
  series: [
    {
      name: 'Heart Rate',
      type: 'line',
      smooth: true,
    }
  ]
});
```

```

    data: records.map((record) => record.heartRate),
  },
],
});

```

3. 在 `src/features/charts/chartRegistry.ts` 注册:

```

{
  id: 'heart-rate',
  title: '心率趋势',
  height: 320,
  buildOption: (dataset) => buildHeartRateOption(dataset.multiparameter),
}

```

4. 页面会自动渲染，也可以在局部组件中直接使用:

```

<BaseChart option={buildHeartRateOption(records)} height={320} />

```

如何新增柱状图

```

interface EventCount {
  name: string;
  count: number;
}

export const buildEventBarOption = (records: EventCount[]) => ({
  tooltip: { trigger: 'axis' },
  xAxis: {
    type: 'category',
    data: records.map((record) => record.name),
  },
  yAxis: {
    type: 'value',
    name: 'count',
  },
  series: [
    {
      name: '事件数量',
      type: 'bar',
      data: records.map((record) => record.count),
    },
  ],
});

```

如何新增饼图

```
interface RatioItem {
  name: string;
  value: number;
}

export const buildRatioPieOption = (records: RatioItem[]) => ({
  tooltip: { trigger: 'item' },
  legend: { bottom: 0 },
  series: [
    {
      name: '占比',
      type: 'pie',
      radius: ['40%', '68%'],
      data: records,
    },
  ],
});
```

如何更新数据

React 中只要 `option` 变化, `BaseChart` 会调用:

```
chartRef.current?.setOption(option, true);
```

所以页面只需要更新数据状态:

```
const [records, setRecords] = useState<HeartRateRecord[]>([]);

return <BaseChart option={buildHeartRateOption(records)} />;
```

上传新文件后:

```
setRecords(parsedRecords);
```

图表会自动刷新。

新增图表类型时的注意事项

本项目为了降低 bundle 体积, 没有 `import * as echarts from 'echarts'`。如果新增散点图, 需要在 `src/charts/BaseChart.tsx` 增加:

```
import { ScatterChart } from 'echarts/charts';
```

```
echarts.use([ScatterChart]);
```

如果新增标题组件、视觉映射、工具箱等，也要从 `echarts/components` 按需导入并注册。